Assignment Kit for Coding Standard

Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview

This assignment kit covers the following topics.

Prerequisites

Prerequisites
- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to
- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements	Produce, document, and submit a completed coding standard that calls for quality coding practices. For LOC counting, ensure that a separate physical source line is used for each logical line of code. Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example
- Since it is an example, tailor it to meet your personal needs.
- If you have an existing organizational standard, consider using it for the PSP exercises.

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	/ **********/ /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ / **********/
Listing Contents	Provide a summary of the listing contents
Contents Example	/ **********/ /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ / **********/

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	/ ********** */ /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ / ********** */
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	If(record_count > limit) /* have all records been processed? */
Bad Comment	If(record_count > limit) /* check if record count exceeds limit */
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	/ ********** */ /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ / ********** */
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.

Indenting Example	while (miss_distance > threshold)
	{
	success_code = move_robot (target _location);
	if (success_code == MOVE_FAILED)
	{
	printf("The robot move has failed.\n");
	}
	}

Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization	#define DEFAULT-NUMBER-OF-STUDENTS 15

Examples	int class-size = DEFAULT-NUMBER-OF-STUDENTS;

Evaluation criteria and suggestions

Evaluation criteria	Your standard must be • complete • legible
Suggestions	Keep your standards simple and short. Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar el desarrollo de programas Java coherentes y fáciles de contar (LOC) y documentar.
Program Headers	Comienza **todos** los archivos con un encabezado descriptivo.

Header Format	`/*`
	` * <ClassName>.java`
	` *`
	` * Copyright (c) 2025 Bridget Mendez. All Rights Reserved.`
	` *`
	` * This software is the confidential and proprietary information of Bridget Mendez`
	` * ("Confidential Information"). You shall not disclose such Confidential Information`
	` * and shall use it only in accordance with the terms of the license agreement you`
	` * entered into with Bridget Mendez.`
	` *`
	` * BRIDGET MENDEZ MAKES NO REPRESENTATIONS OR WARRANTIES ABOUT THE SUITABILITY OF`
	` * THE SOFTWARE, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED`
	` * WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT.`
	` * BRIDGET MENDEZ SHALL NOT BE LIABLE FOR ANY DAMAGES SUFFERED BY LICENSEE AS A`
	` * RESULT OF USING, MODIFYING OR DISTRIBUTING THIS SOFTWARE OR ITS DERIVATIVES.`
	` */`

Listing Contents	Incluye un resumen del contenido del listado.

Contents Example	/
	**********
	*******
	Listing Contents:
	
	  Reuse instructions
	  Modification/Compilation instructions
	  Imports
	  Class declarations:
	  App
	  Logic, Logic2
	  Input, OutPut
	  Data, Media
	  MethodCounter, lineCounter
	  DesvEst
	
	  Source files in ./:
	  App.java -> App.main(String[] args)
	  Logic.java -> Logic.logical()
	  Logic2.java -> Logic2 (WIP)
	  Input.java -> Input.read(String, Data)
	  OutPut.java -> OutPut.writeData(String, String)
	  Data.java -> Data.saveData(String), getAllData(), clearData()
	  Media.java -> Media.calculate(Data)
	  MethodCounter.java -> MethodCounter.count(String[])
	  lineCounter.java -> lineCounter.count()
	  DesvEst.java -> DesvEst.calculate(Data)
	**********
	*******/

Reuse Instructions	• Describe how the program is used. Provide the declaration format, parameter values and types, and parameter limits. • Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.

Reuse Example	/ ********** ******* Reuse instructions Signature: int MethodCounter.count(String[] lines) Purpose: contar firmas de método en código Java (ignora comentarios y blancos) Limits: 'lines' no debe ser null; las firmas pueden abrir bloque en línea siguiente Returns: número total de métodos Signature: int lineCounter.count() Purpose: contar LOC lógicas (excluye //, /*...*/, líneas en blanco) Limits: debe llamarse tras setData(String[]) Signature: void OutPut.writeData(String outFile, String outText) Purpose: escribir el reporte en archivo de salida Warn: sobrescribe el archivo; ruta debe ser válida Signature: void Input.read(String fileName, Data dst) Purpose: leer todas las líneas de un archivo en 'dst' Warn: lanza IOException si no existe el archivo (manejar en Logic) ********** *******/
Identifiers	Use descriptive names for all variables, function names, constants, and other identifiers. Avoid abbreviations or single letter variables.
Identifier Example	Clases/Interfaces en PascalCase: MethodCounter, OutPut, DesvEst. • Métodos/variables en camelCase: logical(), saveData(), lineCount. • Constantes en UPPER_SNAKE_CASE: DEFAULT_BUFFER_SIZE. (Refuerza claridad; una declaración por línea

(continued)

Coding Standard Template (continued)

Comments	• Document the code so that the reader can understand its operation. • Comments should explain both the purpose and behavior of the code. • Comment variable declarations to indicate their purpose.

Good Comment	`if (recordCount > limit) { // check if record count exceeds limit`
	`    ...`
	`}`

Bad Comment	`if (recordCount > limit) { // check if record count exceeds limit`
	`    ...`
	`}`

Major Sections	Precede major program sections by a block comment that describes the processing that is done in the next section

Example	`/`
	`**********`
	`*******`
	` * Lee todos los .java, cuenta LOC y métodos por archivo,`
	` * luego acumula totales y escribe Programa2_ConteoLOC.txt`
	`**********`
	`*******/`

Blank Spaces	• Write programs with sufficient spacing so they do not appear crowded. • Separate every program construct with at least one space.
Indenting	• Indent every level of brace from the previous one. • Open and closing braces should be on lines by themselves and aligned with each other.

Indenting Example	`while (missDistance > threshold) {`
	`    int result = moveRobot(targetLocation);`
	`    if (result == MOVE_FAILED) {`
	`        System.out.println("The robot move has failed.");`
	`    }`
	`}`

Capitalization	• Capitalized all defines. • Lowercase all other identifiers and reserved words. • Messages being output to the user can be mixed-case so as to make a clean user presentation.
Capitalization Example	`static final int DEFAULT_NUMBER_OF_STUDENTS = 15;` `int classSize = DEFAULT_NUMBER_OF_STUDENTS;`

